

Name:- Kartik Fattensing Nalage

PRN :- 202501050052

Batch :-CH3

Practical assignment 4 :

CODETANTRA Home Learn Anywhere 202501050050@mitaoe.ac.in Support Logout

4.1.1. Pandas - series creation and manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5 # Create a Pandas series from the list of numbers
6 df = pd.Series(numbers)
7 # Grouping by even and odd numbers and calculating the mean
8 grouped = df.groupby(df%2==0).mean()
9
10 # Display the mean of even and odd numbers with labels
11 grouped.index = ['Even' if is even else 'Odd' for is even in
```

Average time: 0.011 s, Maximum time: 0.027 s, 3 out of 3 shown test case(s) passed, 3 out of 3 hidden test case(s) passed

Test case 1: Expected output: 1 2 3 4 5 6 7 8 9 10, Mean of even and odd numbers: 0, Odd: 5.0, Even: 6.0, dtype: float64

CODETANTRA Home Learn Anywhere 202501050050@mitaoe.ac.in Support Logout

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column Gender with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to lowercase.

```
1 import pandas as pd
2
3 # Provided dictionary of lists
4 data = {
5     'Name': ['Alice', 'Bob', 'Charlie'],
6     'Age': [25, 30, 35],
7 }
8
9 # Convert the dictionary to a DataFrame
10 df = pd.DataFrame(data)
```

Average time: 0.087 s, Maximum time: 0.098 s, 1 out of 1 shown test case(s) passed, 1 out of 1 hidden test case(s) passed

Test case 1: Expected output: Original DataFrame: Name Age, 0 Alice 25, 1 Bob 30, 2 Charlie 35, New name: Susan, Actual output: Original DataFrame: Name Age, 0 Alice 25, 1 Bob 30, 2 Charlie 35, New name: Susan

CODETANTRA Home Learn Anywhere 202501050050@mitaoe.ac.in Support Logout

4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:

Refer to the displayed test cases for better understanding.

Sample Test Cases

```
1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file, sep="s+", header=None, names=["Name", "Age",
6     "Grade"])
7
8 # write your code here..
9 print("First five rows:")
10 print(data.head())
```

Average time: 0.045 s, Maximum time: 0.065 s, 1 out of 1 shown test case(s) passed, 1 out of 1 hidden test case(s) passed

Test case 1: Expected output: studentdata.txt, First five rows: Name Age Grade, 0 John 25 A, 1 Allin 22 B, 2 Emma 24 A, Actual output: studentdata.txt, First five rows: Name Age Grade, 0 John 25 A, 1 Allin 22 B, 2 Emma 24 A

CODETANTRAHomeLearn Anywhere202501050050@mitaoe.ac.inSupportLogout

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 df['Date'] = pd.to_datetime(df['Date'])
10 df['Month'] = df['Date'].dt.strftime("%Y-%m")
11 df['Total Sales'] = df['Quantity'] * df['Price']
```

Average time: 0.032 s (31.67 ms)Maximum time: 0.064 s (64.00 ms)

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 64 ms Debug

Expected output: sales_data.csv
Actual output: sales_data.csv

Best month: 2025-01
Total sales: \$1210.00

Test cases

CODETANTRAHomeLearn Anywhere202501050050@mitaoe.ac.inSupportLogout

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 product_sales = df.groupby("Product")["Quantity"].sum()
10 # Find the product with the highest total quantity sold
11 best_product = product_sales.idxmax()
```

Average time: 0.023 s (23.33 ms)Maximum time: 0.051 s (51.00 ms)

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 51 ms Debug

Expected output: sales_data.csv
Actual output: sales_data.csv

Best-selling product: Product A
Total quantity sold: 23

Test cases

CODETANTRAHomeLearn Anywhere202501050050@mitaoe.ac.inSupportLogout

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # write the code..
10 city_sales = df.groupby("City")["Quantity"].sum()
11 best_city = city_sales.idxmax()
```

Average time: 0.028 s (27.67 ms)Maximum time: 0.049 s (49.00 ms)

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 49 ms Debug

Expected output: sales_data.csv
Actual output: sales_data.csv

City sold the most products: Los Angeles

Test cases

CODETANTRAHomeLearn Anywhere

202501050050@mitaoe.ac.inSupportLogout

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the following columns: Date, Product, Quantity, Price, and City.

For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).

Output the product pair(s) that was sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	5	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Explanation:

Transactions:

2025-01-01: Product A, Product B

2025-01-02: Product A, Product C

2025-01-03: Product B, Product A

2025-01-04: Product C, Product B

2025-01-05: Product A, Product C

frequentl...

1import pandas as pd

2from itertools import combinations

3from collections import Counter

4

5# Prompt user to input the file name

6file_name = input()

7

8# Read data from the specified CSV file

9df = pd.read_csv(file_name)

10

11# write the code

Average time0.024 s

Maximum time0.047 s

23.67 ms47.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Product A and Product B: 2 times

Product A and Product C: 2 times

Actual output

Product A and Product B: 2 times

Product A and Product C: 2 times

Terminal

Test cases

CODETANTRAHomeLearn Anywhere

202501050050@mitaoe.ac.inSupportLogout

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.

2. Display the last 5 rows of the dataset.

3. Get the shape of the dataset (number of rows and columns).

4. Get a summary of the dataset (using .info()).

5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().

6. Check for missing values and display the count of missing values for each column.

7. Fill missing values in the 'Age' column with the median age.

8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).

9. Drop the 'Cabin' column due to many missing values.

10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

titanicDet...

1import pandas as pd

2import numpy as np

3

4# Load the Titanic dataset

5data = pd.read_csv('Titanic-Dataset.csv')

6

7# 1. Display the first 5 rows of the dataset

8print(data.head())

9

10# 2. Display the last 5 rows of the dataset

11print(data.tail())

Average time0.200 s

Maximum time0.200 s

200.00 ms200.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected output

PassengerId Survived Pclass Fare Cabin Embarked

0 0 1 3 7.2500 S

1 1 2 1 71.2833 C

2 1 3 1 7.9250 S

Actual output

PassengerId Survived Pclass Fare Cabin Embarked

0 0 1 3 7.2500 S

1 1 2 1 71.2833 C

2 1 3 1 7.9250 S

Terminal

Test cases

CODETANTRAHomeLearn Anywhere

202501050050@mitaoe.ac.inSupportLogout

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.

2. Calculate the survival rate by embarkation location (Embarked_S).

3. Calculate the survival rate by family size (FamilySize).

4. Calculate the survival rate by being alone (IsAlone).

5. Get the average fare by passenger class (Pclass).

6. Get the average age by passenger class (Pclass).

7. Get the average age by survival status (Survived).

8. Get the average fare by survival status (Survived).

9. Get the number of survivors by class (Pclass).

10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

titanicDet...

1import pandas as pd

2import numpy as np

3

4# Load the Titanic dataset

5data = pd.read_csv('Titanic-Dataset.csv')

6data['FamilySize'] = data['SibSp'] + data['Parch']

7data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)

8data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

9

10print(data.groupby('Pclass')['Survived'].mean())

11

Average time0.097 s

Maximum time0.097 s

97.00 ms97.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected output

Pclass

1 0.629630

2 0.472826

3 0.242363

Name: Survived, dtype: float64

Embarked_S

0 0.629630

Actual output

Pclass

1 0.629630

2 0.472826

3 0.242363

Name: Survived, dtype: float64

Embarked_S

0 0.629630

Terminal

Test cases

Course

mitaoe.codetantra.com/secure/course.jsp?eucld=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5e/698c54860aba96214c8b7d6a/67e3871c7028471d94db0238

mitaoe.codetantra.com

Home

Learn Anywhere

202501050050@mitaoe.ac.in

Support

Logout

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

00:47

AA

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

Pas sen ger id	Sur vive d	Pcla ss	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Fare	Cab in	Em bark ed

Sample Data:

titanicDat...

Submit

Debugger

1import pandas as pd

2import numpy as np

3

4# Load the Titanic dataset

5data = pd.read_csv('Titanic-Dataset.csv')

6data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

7

8# 1. Get the number of survivors by gender

9survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()

10print(survivors_by_gender)

11

Average time

0.088 s

88.00 ms

Maximum time

0.088 s

88.00 ms

1 out of 1 shown test case(s) passed

Test case 1

68 ms

Debug

Expected output

female:---233

male:-----109

Name: Sex, dtype: int64

male:-----468

female:-----81

Name: Sex, dtype: int64

Actual output

female:---233

male:-----109

Name: Sex, dtype: int64

male:-----468

female:-----81

Name: Sex, dtype: int64

Terminal

Test cases

Prev

Reset

Submit

Next